Chapter 15 — Testing and Debugging

What you will learn. Why automated tests pay for themselves and how to write them: `unittest`, then **pytest** — plain `assert`, fixtures, `parametrize`, `tmp_path`, marks, `conftest.py`. Isolating code from slow or external dependencies with **test doubles** and `unittest.mock` (`Mock`, `patch`, `side_effect`, autospec). **Coverage**, the red-green-refactor **TDD** loop, **doctest**, and **property-based testing** with Hypothesis. Then debugging: tracebacks, `pdb` and `breakpoint()`, logging instead of `print`, and a checklist of the most common Python bugs.

Prerequisites. Functions, classes, exceptions (Chapters 7, 9, 10). Logging basics: Chapter 14.

Code. `chapters/15-testing-and-debugging/code/`: a small module `inventory.py` plus its tests. Install once, run from that folder:

```
pip install pytest pytest-cov hypothesis
cd chapters/15-testing-and-debugging/code
pytest -q                    # 31 passed, 1 skipped, 1 xfailed
python debugging_demo.py
```

("xfailed" = a test *expected* to fail did fail — §15.4.5.)



15.1 Why write tests?

What it is

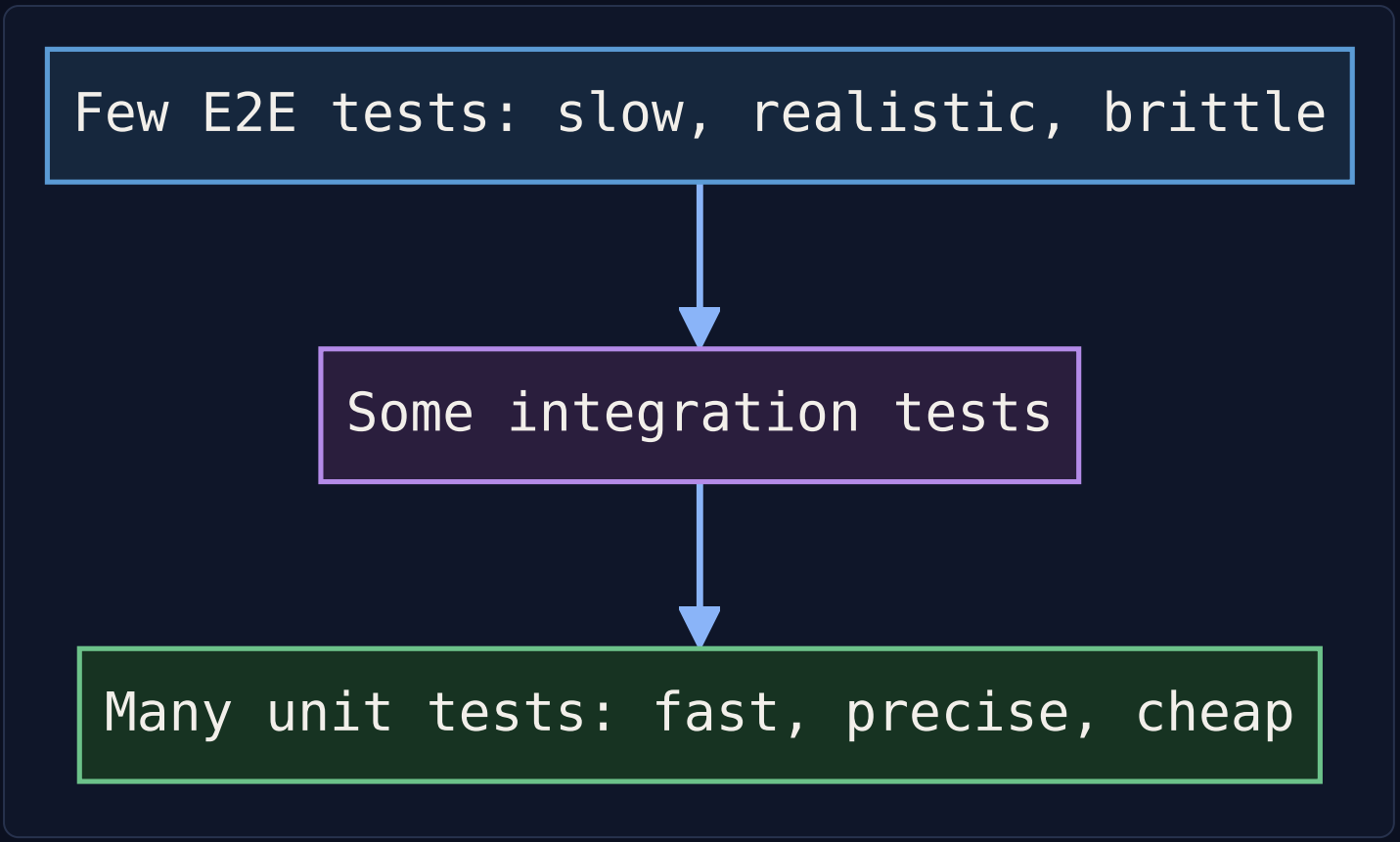
An **automated test** runs your code with known inputs and checks the result.

Term	Meaning
Unit test	tests one small piece (a function, a class) in isolation, in milliseconds
Integration test	tests several pieces together (your code + a real database or file system)
End-to-end (E2E) test	drives the whole system like a user (HTTP calls, browser)
Test suite	all the tests of a project
Regression	a bug that comes back, or a change that breaks something that used to work
SUT	*system under test* — the code being tested
Arrange–Act–Assert	the usual three-part shape of a test: set up, do the thing, check the outcome

Why it matters

1. **Confidence to change code** — refactoring without tests is guessing; with a good suite you know in seconds if anything broke.
2. **Bugs are cheaper earlier** — minutes on your laptop vs incidents, data fixes and lost trust in production.
3. **Executable documentation** — tests show exactly how code is meant to be used.
4. **Better design** — hard-to-test code is usually too coupled; tests push you to small functions with explicit inputs.
5. Python has **no compiler** to catch a typo in a rarely-run branch — only running the code finds it (type checkers help, Chapter 13).

How it works — the test pyramid



Mostly fast unit tests, fewer integration tests, a handful of E2E tests for critical paths — all run automatically in **CI** (continuous integration, e.g. GitHub Actions runs `pytest` on every push and pull request).

What makes a good test

- **Fast** (milliseconds), **isolated** (independent of other tests and order), **repeatable** (no real clock, network or unseeded randomness), **self-checking** (pass/fail, no human reading output), **focused** (one behaviour, named).
- Test **behaviour, not implementation** — what a function returns or changes, not which private helpers it calls — so refactoring doesn't break tests.

Real-world examples

CPython's suite (`python -m test`) uses `unittest` ; NumPy, pandas, pip and most modern projects use pytest; Django ships a `unittest` -based runner. Their pull requests merge only when CI is green.

15.2 The module under test

Every example tests `inventory.py` (abridged — `# ...` marks omitted lines):

```python
class OutOfStock(Exception):
    """Raised when removing more units than are available."""

def slugify(name: str) -> str:
    """Lower-case, keep letters/digits, join words with '-'.

    >>> slugify("Hello, World!")
    'hello-world'
    """
    words = re.findall(r"[a-z0-9]+", name.lower())
    return "-".join(words)

def apply_discount(price: float, percent: float) -> float:
    if not 0 <= percent <= 100:
        raise ValueError(f"percent must be 0-100, got {percent}")
    return round(price * (1 - percent / 100), 2)

@dataclass
class Inventory:
    stock: dict[str, int] = field(default_factory=dict)

    def add(self, sku: str, qty: int) -> None:
        if qty <= 0:
            raise ValueError("qty must be positive")
        self.stock[sku] = self.stock.get(sku, 0) + qty

    def remove(self, sku: str, qty: int) -> None:
        have = self.stock.get(sku, 0)
        if qty > have:
            raise OutOfStock(f"{sku}: want {qty}, have {have}")
        self.stock[sku] = have - qty

    def low_stock(self, threshold: int) -> list[str]: ...   # built with TDD in §15.8
    def total_units(self) -> int: ...
    def save(self, path: Path) -> None:
        path.write_text(json.dumps(self.stock, sort_keys=True))
    @classmethod
    def load(cls, path: Path) -> "Inventory": ...

def price_in(currency: str, usd: float, client: RateClient) -> float:
    """Convert a USD price using an external rate service (mocked in tests)."""
    rate = client.get_rate(currency)
    if rate <= 0:
        raise ValueError("bad rate")
    return round(usd * rate, 2)
```

In `price_in` the rate service is **passed in** (dependency injection) rather than created inside — which makes it trivial to test with a fake (§15.5).

15.3 `unittest` basics

What it is

The standard library's test framework, modelled on Java's JUnit (the "xUnit" family). Tests are methods of a `unittest.TestCase` subclass.

How it works

1. `class TestX(unittest.TestCase)`; each method named `test*` is one test.
2. `setUp()` runs before **each** test, `tearDown()` after each; `setUpClass` / `tearDownClass` (class methods) once per class.
3. **Assertions** (checks that must hold or the test fails): `assertEqual`, `assertTrue`, `assertIn`, `assertIsNone`, `assertAlmostEqual` (7 decimal places by default), `assertRaises` (as a context manager), `assertLogs`.

4. `@unittest.skip(reason)` / `skipIf(cond, reason)` ; `@unittest.expectedFailure` for known bugs.
5. Run: `python -m unittest` (auto-discovers `test*.py`), `python -m unittest -v test_inventory_unittest`, or `unittest.main()` in the file.

```
class TestInventory(unittest.TestCase):
    def setUp(self):                      # runs before EACH test method
        self.inv = Inventory()
        self.inv.add("mug", 5)

    def test_add_accumulates(self):
        self.inv.add("mug", 3)
        self.assertEqual(self.inv.stock["mug"], 8)

    def test_remove_too_many_raises(self):
        with self.assertRaises(OutOfStock):
            self.inv.remove("mug", 99)

    def test_discount(self):
        self.assertAlmostEqual(apply_discount(19.99, 10), 17.99)

    @unittest.skip("demo of skipping")
    def test_skipped(self):
        self.fail("never runs")
```

Output of `python test_inventory_unittest.py`:

```
test_add_accumulates (__main__.TestInventory.test_add_accumulates) ... ok
test_discount (__main__.TestInventory.test_discount) ... ok
test_remove_too_many_raises (__main__.TestInventory.test_remove_too_many_raises) ... ok
test_skipped (__main__.TestInventory.test_skipped) ... skipped 'demo of skipping'

----------------------------------------------------------------------
Ran 4 tests in 0.000s

OK (skipped=1)
```

Trade-offs — unittest vs pytest

	unittest	pytest
Install	built in	`pip install pytest`
Test shape	class + `self.assertX` methods	plain functions + `assert`
Setup	`setUp` / `tearDown`	fixtures (reusable, composable, scoped)
Data-driven tests	`self.subTest(...)` loop	`@pytest.mark.parametrize`
Failure messages	from the assert method	rewritten `assert` shows values and diffs
Plugins (add-on packages)	few	large ecosystem (coverage, xdist for parallel runs, asyncio, django…)
Runs the other's tests	—	yes, pytest runs `unittest` tests unchanged

When to use it

When you can't add dependencies (stdlib-only tools, CPython itself) or in an existing unittest codebase. Otherwise pytest is the default — and it runs unittest tests, so you can migrate gradually.

15.4 pytest

What it is

pytest, the most widely used Python test framework: plain functions with plain `assert`; it discovers, runs and explains failures in detail.

How it works — discovery and running

1. Collects files `test_*.py` / `*_test.py`, functions `test_*`, classes `Test*` (without `__init__`).
2. Common commands:

Command	Does
`pytest`	run everything under the current folder
`pytest -q` / `-v`	quieter / one line per test
`pytest test_x.py::test_name`	one test
`pytest -k "discount and not bad"`	tests whose names match the expression
`pytest -m "not slow"`	select by mark
`pytest -x` / `--maxfail=2`	stop after the first/second failure
`pytest --lf`	rerun only last-failed tests
`pytest -s`	show `print` output (normally captured)
`pytest --pdb`	open the debugger at a failure
`pytest --durations=5`	list the slowest tests

3. Settings: `pyproject.toml` `[tool.pytest.ini_options]` (`testpaths = ["tests"]`, `markers = ["slow: ..."]`, `addopts = "-q"`).

15.4.1 Plain `assert` and assertion rewriting

pytest rewrites `assert` in test files so a failure prints the values and a diff (§15.8 shows one). Helpers:

- `pytest.raises(Exc, match="regex")` — the block must raise `Exc` with a message matching the regex (§14.3).
- `pytest.approx(x)` — float comparison with tolerance (relative 1e-6 default): `0.1 + 0.2 == pytest.approx(0.3)`.
- `pytest.warns(Warning)` — the block must emit a warning.

15.4.2 Fixtures

A **fixture** prepares something a test needs (an object, temp file, DB connection) and optionally cleans up. A test requests it by naming it as a parameter; pytest calls it and passes the result (**dependency injection**).

```python
# conftest.py
@pytest.fixture
def inv():
    """A fresh inventory with two products for every test that asks for it."""
    inventory = Inventory()
    inventory.add("mug", 5)
    inventory.add("pen", 10)
    return inventory

# test_inventory_pytest.py
def test_total_units(inv):
    assert inv.total_units() == 15

def test_remove_reduces_stock(inv):
    inv.remove("pen", 4)
    assert inv.stock == {"mug": 5, "pen": 6}

def test_remove_too_many_raises(inv):
    with pytest.raises(OutOfStock, match="want 6, have 5"):
        inv.remove("mug", 6)
```

1. **Fresh per test by default** (`scope="function"`): `test_remove_reduces_stock` changes its `inv`; the next test still gets the original.
2. **Scopes** (how long one value is shared): `function` (default), `class`, `module`, `package`, `session` (one whole run). Wider scopes for expensive read-only resources (one DB engine per session).
3. **Teardown with `yield`** — code after `yield` runs after the test, even if it fails:

   ```python
   @pytest.fixture
   def audit_log(tmp_path):
       f = (tmp_path / "audit.txt").open("w")
       yield f          # the test runs here
       f.close()        # cleanup
   ```

4. Fixtures can use other fixtures (`audit_log` uses `tmp_path`).
5. `autouse=True` applies a fixture to every test in scope without naming it — sparingly.
6. **Built-in fixtures**: `tmp_path` (unique temp directory as a `Path`), `monkeypatch` (temporarily set attributes, env vars, dict items; auto-undone), `capsys` (capture stdout/stderr), `caplog` (capture log records), `request` (info about the requesting test).

15.4.3 `tmp_path` — real files, safely

```python
def test_save_and_load_roundtrip(inv, tmp_path):
    path = tmp_path / "stock.json"          # unique temp dir per test
    inv.save(path)
    assert Inventory.load(path) == inv
    assert path.read_text() == '{"mug": 5, "pen": 10}'
```

Each test gets its own directory — no collisions, nothing written into your project. Often better than mocking the file system: it tests the real code path and is still fast.

15.4.4 `parametrize` — one test, many cases

```python
@pytest.mark.parametrize(
    ("price", "percent", "expected"),
    [(100, 0, 100), (100, 25, 75), (19.99, 10, 17.99), (50, 100, 0)],
)
def test_discount(price, percent, expected):
    assert apply_discount(price, percent) == pytest.approx(expected)

@pytest.mark.parametrize(("raw", "slug"), [
    ("Blue Mug", "blue-mug"),
    ("  --Hello__World-- ", "hello-world"),
    ("", ""),
], ids=["simple", "punctuation", "empty"])
def test_slugify(raw, slug):
    assert slugify(raw) == slug
```

Each tuple is a **separate test** with its own ID, so one failure doesn't hide others. Include edge cases: empty input, zero, boundaries (0 and 100), negatives, Unicode (`é` , `中` , emoji).

15.4.5 Marks

A **mark** labels a test to change how it's selected or run.

Mark	Effect
`@pytest.mark.skip(reason=...)`	never run
`@pytest.mark.skipif(sys.platform == "win32", reason=...)`	run only when the condition is false
`@pytest.mark.xfail(reason=..., strict=True)`	expected to fail; with `strict=True` , an unexpected *pass* is reported as a failure, so you notice when the bug is fixed
`@pytest.mark.parametrize(...)`	data-driven cases
`@pytest.mark.slow` (custom)	a label you select with `-m` ; register it to avoid "unknown mark" warnings

```python
@pytest.mark.slow
def test_slow_path():
    time.sleep(0.2)
    assert True

@pytest.mark.xfail(reason="float rounding: 2.675 is stored as 2.67499...; use Decimal",
                   strict=True)
def test_known_bug():
    assert apply_discount(2.675, 0) == 2.68
```

The xfail documents a real float issue: `2.675` is stored as slightly less, so `round(2.675, 2)` is `2.67` ; the fix would be `decimal.Decimal` .

15.4.6 `conftest.py`

pytest auto-loads `conftest.py` . Its fixtures and hooks (specially named functions pytest calls, e.g. `pytest_configure`) are available to every test in that directory and below — **no import needed**; nested folders can have their own. Ours defines `inv` and registers `slow` :

```python
def pytest_configure(config):
    config.addinivalue_line("markers", "slow: marks a test as slow")
```

(Real projects register markers in `pyproject.toml` .)

Output

`pytest -v test_inventory_pytest.py` :

```
test_inventory_pytest.py::test_total_units PASSED                          [  6%]
test_inventory_pytest.py::test_remove_reduces_stock PASSED                 [ 12%]
test_inventory_pytest.py::test_remove_too_many_raises PASSED               [ 18%]
test_inventory_pytest.py::test_discount[100-0-100] PASSED                  [ 25%]
...
test_inventory_pytest.py::test_discount_rejects_bad_percent[101] PASSED    [ 56%]
test_inventory_pytest.py::test_slugify[simple] PASSED                      [ 62%]
...
test_inventory_pytest.py::test_slow_path PASSED                            [ 93%]
test_inventory_pytest.py::test_known_bug XFAIL (float rounding: 2.67...) [100%]

======================== 15 passed, 1 xfailed in 0.31s =========================
```

`pytest -q -m "not slow" test_inventory_pytest.py` → `14 passed, 1 deselected, 1 xfailed`. `pytest --durations=3` (whole folder):

```
0.20s call     test_inventory_pytest.py::test_slow_path
0.12s call     test_properties.py::test_stock_never_negative
0.04s call     test_properties.py::test_slugify_is_idempotent
```

Common mistakes

- Tests depending on each other's side effects or order (use fresh fixtures).
- `==` on floats (use `pytest.approx`).
- `pytest.raises` wrapping too much — the exception may come from a setup line.
- Module-level mutable state shared between tests.
- Same-named test files in different folders without `__init__.py` → import-mismatch error.

In the interview

"I use pytest: plain asserts, fixtures for setup with `yield` for teardown, `parametrize` for edge cases, `tmp_path` and `monkeypatch` for files and environment, and marks to separate slow tests. Shared fixtures go in `conftest.py`."

- *Fixture scope?* Per function by default for isolation; session for expensive read-only resources.
- *xfail vs skip?* Skip = don't run; xfail = run, expect failure (strict flags an unexpected pass).

15.5 Test doubles and `unittest.mock`

What it is

A **test double** stands in for a real dependency (a network API, a payment gateway, the clock) during a test — like a stunt double. Standard names (Gerard Meszaros' *xUnit Test Patterns*, popularised by Martin Fowler):

Double	What it does	Example
Dummy	passed but never used	`None` for an unused logger argument
Stub	returns canned (prepared) answers	`get_rate` always returns 83.5
Spy	records how it was called, checked afterwards	a list that collects sent emails
Mock	pre-programmed with expectations about calls, which the test verifies	"`get_rate` must be called once with `'INR'`"
Fake	a working, simplified implementation	an in-memory dict instead of a database

`unittest.mock.Mock` can play stub, spy and mock at once.

Why it matters

Real dependencies make tests **slow** (network), **flaky** (pass/fail with no code change, e.g. service down), **costly** (paid APIs) or **dangerous** (real emails, real charges), and some cases (a timeout on demand) are hard to trigger.

How it works — `Mock`

1. `m = Mock()` — any attribute returns another `Mock`; calls are recorded.
2. `m.get_rate.return_value = 83.5` — **stub** the result.
3. `m.get_rate.side_effect = ...` — an **exception** (raised), an **iterable** (one value per call) or a **function** (called with the same arguments).
4. Verify: `assert_called_once_with(...)`, `assert_called_with`, `assert_not_called`, `call_count`, `call_args`, `call_args_list`.
5. `MagicMock` also supports dunder methods (`__len__`, `__iter__`, `__enter__` …); `AsyncMock` (3.8+) for `async def`.
6. **Autospec** — `create_autospec(RealClass, instance=True)` or `patch(..., autospec=True)` gives a mock with only the real object's attributes and signatures. A plain `Mock` accepts misspelled methods, so a test can pass while real code would crash.

```python
class FakeRates:                              # hand-written fake
    def __init__(self, rates): self.rates = rates
    def get_rate(self, currency): return self.rates[currency]

def test_with_fake():
    assert price_in("EUR", 10, FakeRates({"EUR": 0.9})) == 9.0

def test_with_mock_stub_and_verify():
    client = Mock()
    client.get_rate.return_value = 83.5          # stub the answer
    assert price_in("INR", 2, client) == 167.0
    client.get_rate.assert_called_once_with("INR")   # verify the interaction

def test_side_effect_exception():
    client = Mock()
    client.get_rate.side_effect = TimeoutError("rate service down")
    with pytest.raises(TimeoutError):
        price_in("EUR", 10, client)

def test_side_effect_sequence():
    client = Mock()
    client.get_rate.side_effect = [0.9, -1.0]     # one value per call
    assert price_in("EUR", 10, client) == 9.0
    with pytest.raises(ValueError, match="bad rate"):
        price_in("EUR", 10, client)
    assert client.get_rate.call_args_list == [call("EUR"), call("EUR")]

def test_autospec_catches_typos():
    client = create_autospec(FakeRates, instance=True)
    with pytest.raises(AttributeError):
        client.get_rates("EUR")                   # misspelled: plain Mock would allow it
```

How it works — `patch`

When code creates or imports its dependency itself, `patch` swaps a name **temporarily**:

```python
def test_patch_where_it_is_used(tmp_path):
    # inventory.py did `import json`; patch the name inventory.json.dumps
    with patch("inventory.json.dumps", return_value="{}") as fake_dumps:
        inventory.Inventory({"a": 1}).save(tmp_path / "x.json")
    fake_dumps.assert_called_once()
    assert (tmp_path / "x.json").read_text() == "{}"
```

1. `patch("module.name")` replaces the attribute for a `with` block, a decorated test, or between `start()` / `stop()`; the original is always restored.

2. **Patch where the name is looked up, not where it is defined.** If `shop.py` does `from requests import get`, patch `"shop.get"` — `"requests.get"` won't affect the name already bound in `shop`. (Here `inventory.json` *is* the `json` module, so the patch affects `json.dumps` everywhere during the block — keep patches short.)
3. `patch.object(obj, "attr")` for an object you hold; `patch.dict(os.environ, {...})` for dict entries.
4. pytest's `monkeypatch` covers simple cases: `monkeypatch.setenv("APP_ENV", "test")`, `monkeypatch.setattr(module, "name", value)`.

`pytest -v test_mocking.py` → all 7 tests (the six above plus `test_monkeypatch_env`) `PASSED`.

Trade-offs

Approach	Pros	Cons
Real dependency (tmp_path, SQLite in memory, local server)	tests the real thing	slower, needs setup
Fake	realistic behaviour, reusable, readable	you must write and maintain it
Mock/stub	quick, can simulate errors/timeouts	couples tests to call details; can drift from the real API
`patch`	works without changing the code	fragile if imports move; hints that dependency injection would be cleaner

When not to mock

- What you own and can run cheaply (pure functions, dataclasses, `tmp_path` files, in-memory SQLite).
- The thing you're testing.
- Every internal call — that locks in the implementation. Mock at the **boundary** (network, clock, randomness, third-party SDKs) and prefer checking outcomes over interactions.

Common mistakes

- Patching where defined instead of where used.
- Plain `Mock()` hiding typos — use `autospec` / `spec`.
- Misspelled assertions (`m.called_once_with(...)`, `m.assret_called()`) used to silently "pass" by creating child mocks. Mock now raises `AttributeError` for assertion-like typos (`assret_` etc. since 3.5; names missing the `assert_` prefix since 3.12) unless `unsafe=True`.
- Tests that only prove the mock returns what you told it to.

Real-world examples

`responses` / `respx` fake HTTP for requests / httpx; `freezegun` / `time-machine` freeze the clock; `moto` fakes AWS (Amazon Web Services) services; `fakeredis` fakes Redis (an in-memory key-value store often used as a cache).

In the interview

"I prefer dependency injection so I can pass a fake. At external boundaries I use `unittest.mock` with autospec: `return_value` to stub, `side_effect` for errors or sequences, `assert_called_once_with` only when the interaction matters. With `patch` I patch where the name is looked up."

- *Mock vs stub?* A stub supplies answers; a mock also verifies how it was called.
- *Why autospec?* The mock fails on methods or signatures the real object lacks.
- *`side_effect = [1, 2]`, three calls?* 1, then 2, then `StopIteration`.

15.6 Coverage

What it is

Code coverage measures which lines (and branches) ran during the tests. `coverage.py` is the standard tool; `pytest-cov` integrates it.

- **Line (statement) coverage** — did each line run?
- **Branch coverage** — did each `if` go both ways? A line can run while one side is never taken.

How it works

```
pytest --cov=inventory --cov-branch --cov-report=term-missing
# or without the plugin:
coverage run --branch -m pytest && coverage report -m && coverage html   # htmlcov/index.html
```

Output:

```
Name           Stmts   Miss Branch BrPart  Cover   Missing
-----------------------------------------------------------
inventory.py      48      7     10      2    84%   48, 81-86
-----------------------------------------------------------
TOTAL             48      7     10      2    84%
```

48 statements, 7 never ran; 10 branch points, 2 partially taken. Line **48** is `raise ValueError("qty must be positive")` in `Inventory.add` — no test passes a non-positive quantity, so the validation is untested. **81-86** is the `if __name__ == "__main__":` demo block (exclude with a `# pragma: no cover` comment or config).

Trade-offs / how to use the number

- Coverage shows what is **not** tested, never that tested code is **correct** — a line run without any assertion still "counts".
- Teams often enforce a CI threshold (`--cov-fail-under=85`). 100% is rarely worth it outside small critical libraries; aim high on business logic and error paths.
- In code review, read the *missing* list more than the percentage.
- **Mutation testing** (`mutmut`) goes further: it mutates code (`<` → `<=`) and checks some test fails.

15.7 doctest — examples that are tests

`doctest` finds REPL-style examples in docstrings (`>>> expr` then expected output) and checks that running them produces that output.

```
def slugify(name: str) -> str:
    """Lower-case, keep letters/digits, join words with '-'.

    >>> slugify("Hello, World!")
    'hello-world'
    """
```

- Run: `python -m doctest inventory.py -v`, `doctest.testmod()` in the module, or `pytest --doctest-modules`.
- Output is compared as **text**, so it must match the `repr` exactly.

Output of `python inventory.py` and `pytest --doctest-modules inventory.py`:

```
stock: {'mug': 3} | units: 3
doctest: TestResults(failed=0, attempted=3)
..                                                                       [100%]
2 passed in 0.07s
```

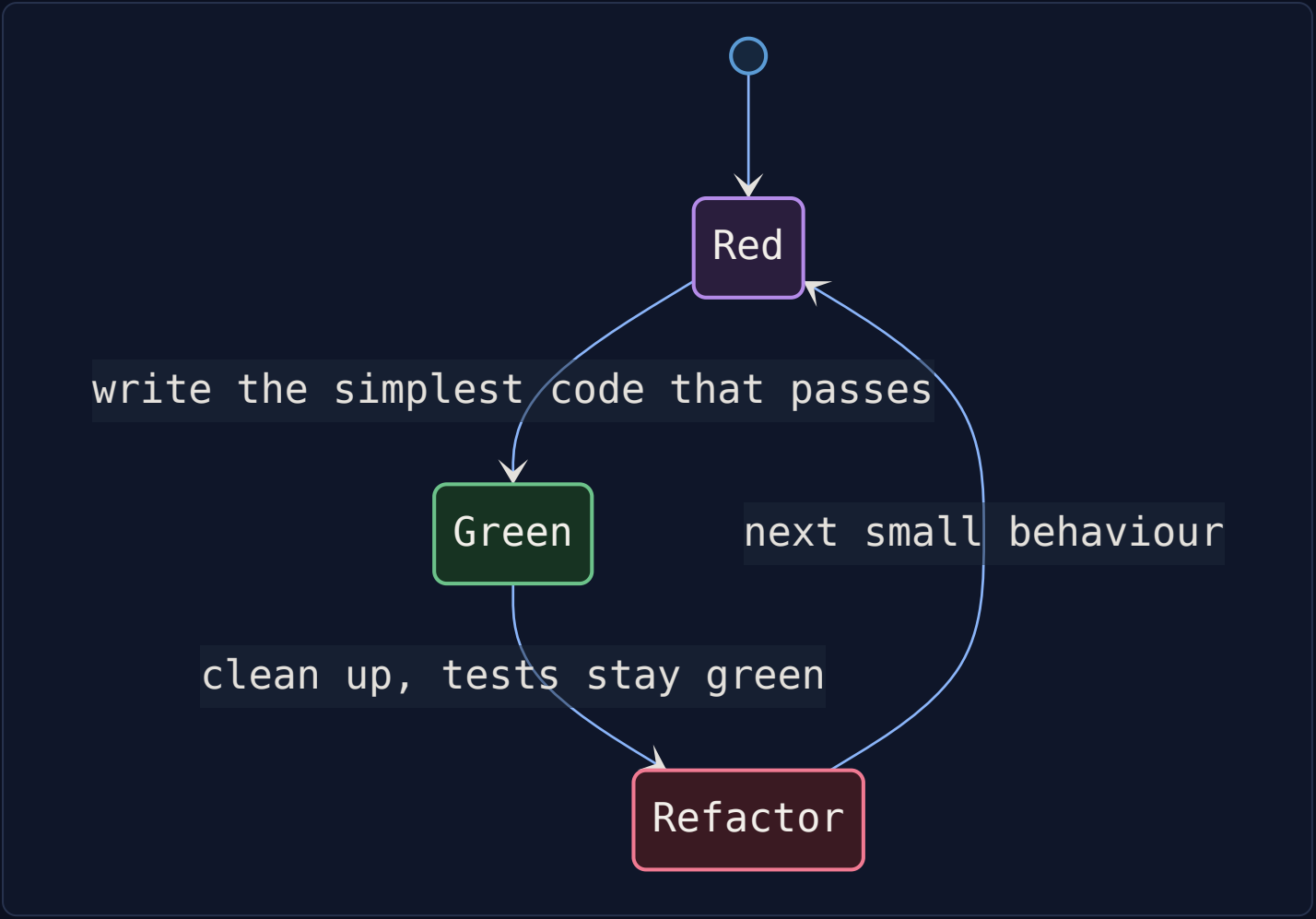
(3 examples in 2 docstrings; pytest counts each docstring as one test.)

Trade-off: great for keeping **documentation examples correct**; poor as a test suite — text comparison is brittle (dict order, float digits, memory addresses) and there are no fixtures or parametrization. Doctests for docs, pytest for tests.

15.8 The TDD loop (red, green, refactor)

What it is

Test-driven development (popularised by Kent Beck): write the test **before** the code, in tiny cycles.



1. **Red** — a small failing test for the next behaviour; see it fail *for the right reason*.
2. **Green** — the minimum code to pass.
3. **Refactor** — improve names and structure under the tests' protection.

How it works — a real cycle

Feature: `Inventory.low_stock(threshold)` → SKUs (stock-keeping units, product codes) with quantity **at or below** the threshold, sorted.

Red. Tests first (`test_tdd_low_stock.py`):

```python
def test_low_stock_lists_skus_below_threshold(inv):      # inv: mug=5, pen=10
    assert inv.low_stock(threshold=6) == ["mug"]

def test_low_stock_sorted_and_inclusive(inv):
    inv.add("cap", 1)
    assert inv.low_stock(threshold=10) == ["cap", "mug", "pen"]
```

Both fail — the method doesn't exist:

```
>       assert inv.low_stock(threshold=6) == ["mug"]
               ^^^^^^^^^^^^^
E       AttributeError: 'Inventory' object has no attribute 'low_stock'
...
2 failed in 0.07s
```

Green, first try. `return [sku for sku, q in self.stock.items() if q < threshold]` passes the first test only; assertion rewriting shows why:

```
>       assert inv.low_stock(threshold=10) == ["cap", "mug", "pen"]
E       AssertionError: assert ['mug', 'cap'] == ['cap', 'mug', 'pen']
E
E         At index 0 diff: 'mug' != 'cap'
E         Right contains one more item: 'pen'
E         Use -v to get more diff
1 failed, 1 passed in 0.07s
```

Two bugs: not sorted, and `pen` (exactly 10) is missing, so the comparison must be `<=`.

Green.

```python
def low_stock(self, threshold: int) -> list[str]:
    """SKUs with quantity <= threshold, alphabetically."""
    return sorted(sku for sku, qty in self.stock.items() if qty <= threshold)
```

Whole suite: `31 passed, 1 skipped, 1 xfailed`.

Refactor. Names are already clear — next behaviour.

Trade-offs

Pros	Cons
Every line has a reason (a test)	Slower start; feels unnatural at first
Forces small, testable design	Hard when exploring an unknown problem/API (spike — a throw-away experiment — first, then test)
Instant regression safety	Can over-fit tests to one design if cycles are too big

Many teams use TDD at least for bug fixes: **first write a test that reproduces the bug**, see it fail, then fix — so it can never silently return.

15.9 Property-based testing with Hypothesis (brief)

What it is

In an **example-based** test you pick inputs. In a **property-based** test you state a rule that must hold for *all* inputs of a kind, and the library generates hundreds — including nasty ones (empty strings, huge numbers, Unicode, NaN) — to break it. **Hypothesis** is Python's main library (the idea comes from Haskell's QuickCheck). A failing input is **shrunk** to the simplest failing example and saved in `.hypothesis/` so it's retried first next time.

How it works

```python
from hypothesis import given, settings, strategies as st

@given(st.text())
def test_slugify_is_idempotent(s):            # applying it twice changes nothing
    assert slugify(slugify(s)) == slugify(s)

@given(st.text())
def test_slug_has_only_safe_chars(s):
    assert all(c.isascii() and (c.isalnum() or c == "-") for c in slugify(s))

@given(price=st.floats(0, 1e6), pct=st.floats(0, 100))
def test_discount_never_increases_price(price, pct):
    assert apply_discount(price, pct) <= round(price, 2)

@settings(max_examples=200)
@given(st.lists(st.tuples(st.sampled_from(["add", "remove"]), st.integers(1, 20))))
def test_stock_never_negative(ops):           # random sequences of operations
    inv = Inventory()
    for op, qty in ops:
        try:
            getattr(inv, op)("mug", qty)
        except OutOfStock:
            pass
        assert inv.stock.get("mug", 0) >= 0
```

All four pass. Against a naive `name.strip().lower().replace(" ", "-")`, the "safe characters" property fails and shrinks to one character:

```
E       Failing test case: test_slug_has_only_safe_chars(
E           s=':',
E       )
```

Common properties: **round-trip** (`load(save(x)) == x`, `decode(encode(x)) == x`), **idempotence** (`f(f(x)) == f(x)`), **invariants** (stock never negative, output sorted, length preserved), **oracle** (fast version equals a slow obviously-correct one). Great for parsers and serializers.

When not to use it: no general rule to state, or inputs expensive to build. Use alongside example tests, not instead.

15.10 Reading tracebacks

What it is

A **traceback** reports an uncaught exception: the **call stack** that led to it, oldest call at the top, failing line at the bottom.

How it works — read it bottom-up

1. **Last line first** — type and message: `ValueError: invalid literal for int() with base 10: 'two'`.
2. **The frame above it** (each entry is a *frame* — one function call in progress): file, line, function. Since 3.11, `~~~^^^` markers point at the exact sub-expression (PEP 657).
3. **Walk upward** to *your* code (skip library frames) — often the bug is where your code passed bad data in.
4. **Chained exceptions**: "The above exception was the direct cause of the following exception" = explicit `raise NewError(...) from original`. "During handling of the above exception, another exception occurred" = a new error *inside* an `except` block (implicit chaining).

From `debugging_demo.py` (`"two"` as a quantity):

```
Traceback (most recent call last):
  File "debugging_demo.py", line 19, in load_orders
    orders.append(parse_order(line))
                  ~~~~~~~~~~~^^^^^^
  File "debugging_demo.py", line 12, in parse_order
    return {"sku": sku, "qty": int(qty), "price": float(price)}
                               ~~~^^^^^
ValueError: invalid literal for int() with base 10: 'two'

The above exception was the direct cause of the following exception:

Traceback (most recent call last):
  File "debugging_demo.py", line 35, in <module>
    load_orders(["mug,2,4.50", "pen,two,1.20"])
    ~~~~~~~~~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "debugging_demo.py", line 21, in load_orders
    raise RuntimeError(f"bad order on line {n}: {line!r}") from e
RuntimeError: bad order on line 2: 'pen,two,1.20'
```

The final error says *which input* was bad; the cause says *what* was wrong (`int('two')`) — the payoff of `raise ... from e` with context.

Common exception → usual cause

Exception	Usual cause
`NameError`	typo, or used before assignment
`AttributeError: 'NoneType' object has no attribute ...`	a function returned `None` (forgot `return`, or an in-place method like `list.sort()`)
`TypeError: ... takes 2 positional arguments but 3 were given`	forgot `self`, or wrong call
`KeyError` / `IndexError`	missing dict key / off-by-one
`UnboundLocalError`	assigning to a name inside a function makes it local (use `nonlocal` / `global` or rename)
`ModuleNotFoundError`	wrong virtual environment (venv), or a local file shadows the module (your own `random.py`)
`RecursionError`	missing or unreachable base case

15.11 Debugging with `pdb` and `breakpoint()`

What it is

`pdb` is Python's interactive **debugger**: pause at a line, inspect variables, step, move up and down the stack. `breakpoint()` (3.7+, PEP 553) pauses there — it calls `pdb.set_trace()` by default.

How it works — ways in

Way	When
Put `breakpoint()` in the code	pause at a specific spot
`python -m pdb script.py`	start the whole script under the debugger
`pdb.pm()` after a crash (in the REPL) or `python -m pdb` (enters post-mortem — inspecting the state at the moment of the crash — on an uncaught exception)	inspect the frame where it died
`pytest --pdb`	debugger at the failing assertion
`pytest --trace`	debugger at the start of each test
IDE (integrated development environment: VS Code, PyCharm) breakpoints	same ideas, with a GUI

`PYTHONBREAKPOINT` controls `breakpoint()`: `=0` disables all (safety net in CI), `=ipdb.set_trace` switches debugger.

The commands you need

Command	Does
`l` / `ll`	list code around the current line / the whole function
`n` (next)	run the current line, stepping *over* calls
`s` (step)	step *into* a call
`r` (return)	run until the current function returns
`c` (continue)	run until the next breakpoint
`unt N` (until)	run until line N
`p expr` / `pp expr`	print / pretty-print an expression
`w` (where)	show the call stack
`u` / `d`	move up / down the stack to inspect callers
`b file:line` / `b func` / `b line, cond`	set a (conditional) breakpoint
`cl N`	clear breakpoint N
`display expr`	show `expr` each time execution stops if it changed
`interact`	a full Python REPL in the current scope
`q`	quit

A bare expression works too, but for variables named like commands (`n`, `c`, `l`…) use `p n`.

A real session (scripted, so it can be shown)

`debugging_demo.py` feeds `pdb` a command list. The function crashes on an empty list:

```python
def average_value(orders: list[dict]) -> float:
    total = 0.0
    for o in orders:
        total += o["qty"] * o["price"]
    return total / len(orders)          # bug when orders is empty
```

Commands: `b average_value`, `c`, `p orders`, `n`, `n`, `p total, len(orders)`, `c`. Transcript:

```
(Pdb) Breakpoint 1 at debugging_demo.py:26
(Pdb) > debugging_demo.py(26)average_value()
-> total = 0.0
(Pdb) []
(Pdb) > debugging_demo.py(27)average_value()
-> for o in orders:
(Pdb) > debugging_demo.py(29)average_value()
-> return total / len(orders)          # bug when orders is empty
(Pdb) (0.0, 0)
(Pdb)
then it raised: ZeroDivisionError('division by zero')
post-mortem locals: {'orders': [], 'total': 0.0}
breakpoint() skipped because PYTHONBREAKPOINT=0
```

The loop body is skipped and `len(orders)` is 0 at the return — handle the empty case (`if not orders: return 0.0` or raise a clear error). "post-mortem locals" is the same information recovered from the innermost traceback frame after the crash — what `pdb.pm()` gives interactively.

A debugging method (not just a tool)

1. **Reproduce** reliably — ideally as a failing test.
2. **Read** the traceback and code; state a hypothesis.
3. **Observe** with `breakpoint()`, logging or `p`.
4. **Narrow down**: halve the input, comment out half the code, or `git bisect` (binary-searches history for the commit that introduced the bug).
5. **Fix** the cause, not the symptom; keep the test.
6. Explaining the code line by line ("rubber-duck debugging") often reveals the bug.

Common mistakes

- Committing a `breakpoint()` — CI hangs waiting for input. Set `PYTHONBREAKPOINT=0` there and use a linter rule such as Ruff's `T100`.
- Changing several things at once.

15.12 Logging vs `print` for debugging

`print` is fine for a five-minute look. Anything that stays in the code uses **logging** (Chapter 14) — levels, timestamps, module names, configurable destinations, DEBUG enabled only where needed.

	`print`	`logging`
Levels / filtering	no	DEBUG…CRITICAL, per module
Timestamp, module, line	manual	via formatter
Destination	stdout	console, files, syslog (the OS's central log), log services
Safe to leave in code	no	yes (DEBUG off in production)
In tests	captured by pytest (`-s` shows)	`caplog` fixture lets tests assert on logs

```
log = logging.getLogger(__name__)
log.debug("pricing order=%s rate=%s", order_id, rate)    # visible only at DEBUG
```

Tips: raise one module's level temporarily (`logging.getLogger("shop.pricing").setLevel(logging.DEBUG)`); `print(f"{total=}")` prints `total=0.0`; `python -X dev` enables extra runtime checks and warnings; `faulthandler` (`python -X faulthandler`) dumps tracebacks on hard crashes (e.g. a segfault in C code) or hangs.

15.13 Common bugs checklist

Run through this when something "can't possibly be wrong":

#	Bug	Example	Fix
1	Mutable default argument	`def f(items=[])` shares one list across calls	`items=None` then `items = [] if items is None else items`
2	Late-binding closures	`[lambda: i for i in range(3)]` all return 2	`lambda i=i: i`
3	Aliasing / shallow copies	`grid = [[0]*3]*3` — one row shared three times	`[[0]*3 for _ in range(3)]`, `copy.deepcopy`
4	Modifying a list while iterating	removing items inside `for x in xs` skips items	iterate over a copy or build a new list
5	`is` vs `==`	`x is 1000` depends on object identity	`==` for values; `is` only for `None`, sentinels (unique marker objects)
6	Float equality	`0.1 + 0.2 == 0.3` is False	`math.isclose`, `pytest.approx`, `Decimal` for money
7	Integer vs float division	`7 / 2 == 3.5`, `7 // 2 == 3`, `-7 // 2 == -4`	pick the right operator
8	Methods returning `None`	`xs = xs.sort()` makes `xs` None	`xs.sort()` or `xs = sorted(xs)`
9	Swallowed exceptions	`except Exception: pass` hides the real error	catch specific exceptions, log, re-raise
10	Off-by-one	`range(1, n)` excludes `n`	test the boundaries
11	Truthiness	`if count:` is False for `0`	`if count is not None:`
12	Naive datetimes / time zones	comparing local and UTC times	aware UTC datetimes (Chapter 14)
13	Unclosed resources	files/connections left open	`with` statements
14	Wrong environment	`ModuleNotFoundError` though "it's installed"	check `sys.executable`, activate the venv, `python -m pip`
15	Module name shadowing	your `random.py` or `test.py` hides the stdlib	rename the file
16	Global/shared state between tests	tests pass alone, fail together	fresh fixtures, no module-level mutable state
17	Encoding	`UnicodeDecodeError` reading files	`open(p, encoding="utf-8")`
18	Unseeded randomness / real clock in tests	flaky tests	seed the generator, inject the clock, freeze time

---

15.14 Chapter self-check

1. Three parts of a well-shaped test? → Arrange, Act, Assert.
2. Why do most tests belong at the bottom of the pyramid? → Unit tests are fast, precise, cheap.
3. How does pytest show values in a failing `assert`? → Assertion rewriting.
4. How does a test get a fixture, and must it import from `conftest.py`? → By naming it as a parameter; no, discovered automatically.

5. Fixture cleanup? → Code after `yield`.
6. `parametrize` with four tuples produces? → Four separate tests.
7. What does `tmp_path` give you? → A unique temporary directory (`Path`) per test.
8. `xfail(strict=True)` and the test passes? → Reported as a failure (XPASS strict), so you notice the fix.
9. Stub vs mock vs fake? → Canned answers / verified interactions / working simplified implementation (e.g. an in-memory repository).
10. `shop.py` has `from requests import get` — what do you patch? → `"shop.get"`.
11. Why `create_autospec`? → Mocks reject attributes and signatures the real object lacks.
12. What does 100% line coverage prove? → Only that every line ran — not that results were checked.
13. TDD cycle? → Red, green, refactor.
14. What does Hypothesis do after finding a failing input? → Shrinks it to a minimal example and remembers it.
15. How do you read a traceback? → Bottom-up: message, failing line, then up to your code; "direct cause" = `from`, "during handling" = error inside `except`.
16. Disable every `breakpoint()` in CI? → `PYTHONBREAKPOINT=0`.
17. `xs = xs.sort()` — what is `xs`? → `None`.

Chapter TL;DR

- **Why:** tests make change safe. Many fast unit tests, fewer integration/E2E; deterministic, isolated, behaviour-focused; run in CI.
- **unittest:** `TestCase`, `setUp`, `assertEqual` / `assertRaises`; built in, more verbose.
- **pytest:** plain `assert` (rewritten failure diffs), fixtures (`yield` teardown, scopes, `conftest.py`), `parametrize`, `tmp_path`, `monkeypatch`, `capsys`, `caplog`, marks (`skip`, `skipif`, `xfail`, custom + `-m`), `-k`, `-x`, `--lf`, `--pdb`.
- **Doubles:** dummy, stub, spy, mock, fake. `Mock` + `return_value` / `side_effect` / `assert_called_once_with`; `patch` where the name is used; autospec; mock only at boundaries; prefer dependency injection.
- **Coverage:** `pytest --cov --cov-branch --cov-report=term-missing` finds untested code; it doesn't prove correctness.
- **TDD:** red, green, refactor; always reproduce a bug with a failing test first.
- **doctest** keeps doc examples true; **Hypothesis** checks properties and shrinks counter-examples.
- **Debugging:** read tracebacks bottom-up; `breakpoint()` / `pdb` (`n s c p w u d b`), `pytest --pdb`, post-mortem; reproduce → hypothesise → observe → narrow → fix → keep the test.
- **Logging over print** for anything that stays; check the **common bugs list** first (mutable defaults, aliasing, `None` returns, float equality, swallowed exceptions, wrong venv).